

Estructuras de Datos y Algoritmos

Tema 3: Elementos de corrección algorítmica

Dr. Jorge Armando Ramírez Díaz

Universidad Nacional Autónoma de México
ENES Unidad Juriquilla
Licenciatura en Matemáticas para el Desarrollo

¿Qué significa que un algoritmo sea correcto?

Cada algoritmo se diseña para cumplir una tarea específica.

Diremos que es correcto si, para cada entrada permitida por su especificación, termina y produce una salida que cumple lo especificado.

La corrección se establece **respecto de una especificación**: debemos precisar las entradas admitidas y los resultados esperados.

Es común escribir algoritmos mediante una combinación de análisis lógico y prueba y error, como ya lo hemos estado haciendo.

Durante ese proceso:

- Corregimos los errores de sintaxis que impiden ejecutar el programa.
- Buscamos y corregimos errores de lógica.
- Probamos entradas para las que conocemos la salida esperada.

¿Con esto tenemos plena seguridad de que el algoritmo es correcto para todas las entradas permitidas?

En 1967, Robert W. Floyd publicó un trabajo fundamental para el desarrollo de métodos de demostración de corrección de programas.

Edsger W. Dijkstra

«Adoptamos ahora la postura de que la tarea del programador no consiste únicamente en producir un programa correcto, sino también en demostrar de manera convincente su corrección».

David Gries

«Un programa y su demostración de corrección deben desarrollarse de manera conjunta, y normalmente es la demostración la que guía el proceso».

Traducciones de las citas recogidas por Epp, sección 5.5, p. 315.

Pruebas y demostraciones

En Fibonacci comprobamos casos como:

```
assert fibonacci_iterativo(0) == 0
assert fibonacci_iterativo(10) == 55
assert fibonacci_iterativo(30) == 832040
```

Esos resultados aportan evidencia sobre las entradas probadas.

- Un contraejemplo basta para demostrar que el algoritmo es incorrecto.
- Pasar una selección de pruebas no demuestra que funcione para todos los enteros permitidos.
- Una demostración debe justificar el comportamiento para una entrada arbitraria del dominio.

Definición: estado

Asignación de un valor actual a cada variable del programa.

Por ejemplo, en el estado $x = 3, y = 6$:

$$x \geq 0 \quad y \quad y = 2x$$

son afirmaciones verdaderas.

Después de ejecutar $x \leftarrow x + 1$, el estado es $x = 4, y = 6$. Ahora $x \geq 0$ sigue siendo verdadera, pero $y = 2x$ es falsa.

Para las reglas siguientes usaremos variables enteras, aritmética exacta y expresiones bien definidas cuya evaluación no modifica variables.

Precondición y postcondición

Definición: precondición

Propiedad que suponemos verdadera antes de ejecutar el algoritmo.

Definición: postcondición

Propiedad que debe cumplirse al terminar el algoritmo.

División entera:

$$P: \quad a, d \in \mathbb{Z}, \quad a \geq 0, \quad d > 0.$$

$$Q: \quad q, r \in \mathbb{Z}_{\geq 0}, \quad a = qd + r, \quad 0 \leq r < d.$$

Suponemos que a, d conservan los valores de entrada. La igualdad $a = qd + r$ no basta: también necesitamos $0 \leq r < d$.

Corrección parcial y total

Definición: corrección parcial

Para toda entrada que satisface la precondition, si el algoritmo termina, entonces su estado final satisface la postcondición.

Definición: corrección total

Para toda entrada que satisface la precondition, el algoritmo termina y su estado final satisface la postcondición.

$\text{Corrección total} = \text{corrección parcial} + \text{terminación}.$

La corrección parcial no garantiza que la ejecución termine.

Tripletas de Hoare

Definición: tripleta de Hoare

Una especificación formada por una precondition P , un fragmento de código C y una postcondición Q :

$$\{ P \} C \{ Q \}.$$

En esta presentación, la tripleta expresa **corrección parcial**. Es válida si, desde cualquier estado que satisface P , toda ejecución de C que termina deja un estado que satisface Q .

$$\{ x < 4 \} x \leftarrow x + 1 \{ x < 5 \}.$$

Estado inicial: $x = 1$, que satisface $x < 4$.

Instrucción: $x \leftarrow x + 1$.

Estado final: $x = 2$, que satisface $x < 5$.

Esta ejecución es un caso particular. La tripleta debe valer para todos los estados que satisfacen P . Si P no se cumple, la tripleta no establece una garantía.

Definición: aserción

Afirmación sobre los valores de las variables en un punto concreto de la ejecución.

Instrucción	Aserción después de ejecutarla
Entrada $n \in \mathbb{Z}_{\geq 0}$	$n \geq 0$
$x \leftarrow n + 1$	$x = n + 1$
$y \leftarrow 2x$	$y = 2(n + 1)$

Justificamos cada transición usando el estado anterior.

Un `assert` de Python comprueba una aserción en una ejecución; la demostración explica por qué vale para todas las entradas admitidas.

Intercalamos afirmaciones con las instrucciones del algoritmo:

$\{A_1\}$	Precondición del algoritmo
C_1	Primer fragmento de instrucciones
$\{A_2\}$	Aserción intermedia
C_2	Siguiente fragmento de instrucciones
$\vdots$	$\vdots$
$\{A_{k-1}\}$	Última aserción intermedia
C_{k-1}	Último fragmento de instrucciones
$\{A_k\}$	Postcondición del algoritmo

Cada par de aserciones **sucesivas** funciona como precondición y postcondición del fragmento situado entre ellas.

Para cada $i = 1, 2, \dots, k - 1$, debemos demostrar:

$$\{ A_i \} C_i \{ A_{i+1} \}.$$

Si A_i es verdadera y el fragmento C_i termina, entonces A_{i+1} debe ser verdadera.

Debemos justificar **todas las transiciones**, incluida la que va de A_{k-1} a A_k .

Si A_1 es la precondition y A_k la postcondición, estas pruebas permiten concluir la corrección parcial de la secuencia completa.

Para concluir corrección total también debemos justificar su terminación.

La instrucción

$$x \leftarrow E$$

evalúa E con los valores del estado anterior y guarda el resultado en x . Las demás variables conservan su valor.

Para $x \leftarrow x + 1$, la relación entre estados es

$$x_{\text{nuevo}} = x_{\text{anterior}} + 1.$$

No afirmamos $x = x + 1$ dentro de un mismo estado.

En pseudocódigo usaremos $\leftarrow$ para asignar y $=$ para igualdad matemática. En Python se asigna con $=$ y se compara con $==$.

Definición: expresión

Combinación de valores, variables y operaciones que se evalúa para obtener un valor, como $x + 1$ o $2x + y$.

Bien definida: puede evaluarse para los valores considerados. Por ejemplo, $x + 1$ está definida para cualquier entero x ; en cambio, $1/x$ requiere $x \neq 0$.

Sin efectos secundarios: evaluarla no modifica el estado del programa.

Si $x = 4$, evaluar $x + 1$ produce 5, pero x sigue valiendo 4. Al ejecutar $x \leftarrow x + 1$, primero evaluamos la expresión y después guardamos el resultado: ahora $x = 5$.

Definición: sustitución

$Q[E/x]$ se lee «la fórmula Q , sustituyendo x por E ». La barra no representa una división.

En nuestros primeros ejemplos usaremos aserciones sin cuantificadores: sustituimos todas las apariciones de x .

$$\underbrace{x > 5}_Q \xrightarrow{\text{sustituir } x \text{ por } x+1} \underbrace{x + 1 > 5}_{Q[E/x]}.$$

Transformamos una fórmula; todavía no ejecutamos una asignación.

Una aparición de x es **libre** si no está ligada por un cuantificador: en $x > 5$, x es libre; en $\forall x (x^2 \geq 0)$, está ligada por $\forall x$. La regla de asignación sustituye las apariciones libres.

Regla de asignación

Regla de asignación

Para una expresión E bien definida y sin efectos secundarios:

$$\{ Q[E/x] \} x \leftarrow E \{ Q \}.$$

Queremos que Q se cumpla después de asignar. Antes debe cumplirse $Q[E/x]$, porque x recibirá el valor de E .

Por ejemplo, si Q es $x > 5$ y E es $x + 1$:

$$Q[E/x] \text{ es } x + 1 > 5.$$

$$\{ x + 1 > 5 \} x \leftarrow x + 1 \{ x > 5 \}.$$

Se parte de lo que queremos al final para obtener una condición suficiente antes de la asignación. Aquí puede simplificarse a $x > 4$.

Validez de la regla de asignación.

- 1 Supongamos que el estado inicial satisface $Q[E/x]$.
- 2 La instrucción $x \leftarrow E$ coloca en x el valor de E calculado en ese estado.
- 3 Todas las demás variables conservan su valor.
- 4 Por la sustitución, evaluar Q en el estado final equivale a evaluar $Q[E/x]$ en el estado inicial. Por tanto, se cumple Q .

La regla se aplica a una asignación concreta. Si hay instrucciones intermedias, también hay que justificar sus efectos.

Ejemplo: asignación

Queremos ejecutar $x \leftarrow x + 1$ y garantizar que después se cumpla $x > 5$.

Valor anterior y valor nuevo

Sea a el valor de x antes de la asignación.

- 1 Antes, x vale a .
- 2 Calculamos $a + 1$.
- 3 Guardamos el resultado: ahora x vale $a + 1$.

Condición necesaria antes

Para que el valor nuevo sea mayor que cinco, necesitamos:

$$a + 1 > 5 \iff a > 4.$$

En la regla, Q es $x > 5$ y E es $x + 1$. Al sustituir obtenemos $Q[E/x] : x + 1 > 5$.

$$\boxed{\{x > 4\} x \leftarrow x + 1 \{x > 5\}}.$$

Si $x = 5$, obtenemos $x = 6$, que cumple la postcondición. Si $x = 4$, obtenemos $x = 5$, que no cumple $x > 5$.

La precondition habla del valor anterior; la postcondición, del nuevo.

Teorema de composición secuencial

Si las tripletas

$$\{P\} C_1 \{R\} \quad \text{y} \quad \{R\} C_2 \{Q\}$$

son válidas, entonces también lo es

$$\{P\} C_1; C_2 \{Q\}.$$

$C_1; C_2$ significa ejecutar primero C_1 y después C_2 .

La aserción R conecta ambos fragmentos: es la postcondición del primero y la precondition del segundo.

Composición secuencial. Supongamos que P es verdadera y que la secuencia $C_1; C_2$ termina.

- ➊ Al terminar C_1 , se cumple R , por la primera tripleta.
- ➋ Ese estado es el estado inicial de C_2 .
- ➌ Al terminar C_2 , se cumple Q , por la segunda tripleta.

Por tanto, $\{ P \} C_1; C_2 \{ Q \}$ es válida. El argumento se extiende a una secuencia finita de fragmentos.

Si cada fragmento, además, termina bajo su precondition correspondiente, obtenemos corrección total de la secuencia.

Regla del condicional

Si son válidas

$$\{P \wedge B\} C_1 \{Q\}, \quad \{P \wedge \neg B\} C_2 \{Q\},$$

entonces es válida

$$\{P\} \text{ si } B \text{ entonces } C_1; \text{ si no, } C_2 \{Q\}.$$

B es una condición booleana bien definida.

- En la primera rama podemos usar P y B .
- En la segunda podemos usar P y $\neg B$.
- Las dos ramas deben establecer la misma postcondición Q .

Validez de la regla del condicional. Supongamos que P es verdadera y que el condicional termina.

Caso 1: B es verdadera.

Se ejecuta C_1 desde un estado que satisface $P \wedge B$. Su tripleta garantiza Q al terminar.

Caso 2: B es falsa.

Se ejecuta C_2 desde un estado que satisface $P \wedge \neg B$. Su tripleta también garantiza Q al terminar.

Los casos cubren todas las posibilidades. Por tanto, el condicional satisface Q siempre que termina desde un estado que cumple P .

Ejemplo: condicionales

Queremos calcular el valor absoluto de un entero:

$$P : x \in \mathbb{Z}, \quad B : x \geq 0, \quad Q : y = |x|.$$

pseudocódigo

Si $x \geq 0$:

$$y \leftarrow x \quad (C_1)$$

En otro caso:

$$y \leftarrow -x \quad (C_2)$$

El valor de x no se modifica.

Las dos premisas permiten aplicar la regla del condicional:

$$\frac{\{P \wedge B\} C_1 \{Q\} \quad \{P \wedge \neg B\} C_2 \{Q\}}{\{P\} \text{ si } B \text{ entonces } C_1; \text{ si no, } C_2 \{Q\}}.$$

Primera premisa:

$$\{P \wedge B\} y \leftarrow x \{Q\}.$$

Bajo $P \wedge B$, $x \geq 0$, por lo que $|x| = x$. La asignación establece Q .

Segunda premisa:

$$\{P \wedge \neg B\} y \leftarrow -x \{Q\}.$$

Bajo $P \wedge \neg B$, $x < 0$, por lo que $|x| = -x$. Esta asignación también establece Q .

Estructura de un ciclo

$\{P\}$: precondition del ciclo

Repetir mientras G :

 Ejecutar el cuerpo C

$\{Q\}$: postcondición del ciclo

La condición G se llama **guarda**. Se evalúa antes de cada posible iteración.

- Si G es verdadera, se ejecuta el cuerpo.
- Si G es falsa, se sale del ciclo.
- El cuerpo puede ejecutarse cero veces.

Consideramos ciclos cuya salida ocurre al hacerse falsa la guarda, sin `break` ni retornos internos.

Corrección de un ciclo

Definición: corrección parcial de un ciclo

Un ciclo es parcialmente correcto respecto de sus precondiciones y postcondiciones si y solo si, siempre que el estado inicial satisface la precondición y el ciclo termina tras un número finito de pasos, el estado final satisface la postcondición.

En la notación de Hoare:

$$\{ P \} \text{ mientras } G \text{ ejecutar } C \{ Q \}.$$

La condición P describe el estado justo antes del ciclo, después de las instrucciones de inicialización.

Para corrección total también necesitamos que el ciclo termine.

Definición: invariante de ciclo

Un predicado I sobre el estado que se conserva al ejecutar el cuerpo bajo la guarda:

$$\{I \wedge G\} C \{I\}.$$

Si I y G se cumplen antes de una iteración y el cuerpo termina, I vuelve a cumplirse después.

Para usarlo en la prueba también debemos establecer I antes de la primera evaluación de la guarda.

Lo observamos antes de evaluar la guarda y al completar el cuerpo; puede fallar temporalmente entre instrucciones internas.

Sea $I(k)$ la afirmación de que el estado después de k iteraciones completas satisface el invariante I .

- **Base:** $I(0)$ se cumple antes de la primera iteración.
- **Paso inductivo:** para cada entero $k \geq 0$, si se cumplen $I(k)$ y la guarda, una iteración completa establece $I(k + 1)$.

Por inducción, el invariante vale después de cada número de iteraciones que se ejecuta.

k cuenta iteraciones en la demostración; no necesita ser una variable del programa. Este argumento no demuestra terminación.

Demostración

Retomamos $I(k) : p_k + r_k x = mx, r_k \in \mathbb{Z}_{\geq 0}$.

Propiedad inductiva. Si $I(k)$ y la guarda $r_k > 0$ son verdaderas, el cuerpo establece:

$$p_{k+1} = p_k + x, \quad r_{k+1} = r_k - 1 \geq 0.$$

Por tanto:

$$p_{k+1} + r_{k+1}x = (p_k + x) + (r_k - 1)x = p_k + r_k x = mx.$$

Así se cumple $I(k+1)$.

Eventual falsedad de la guarda.

Cada iteración termina y reduce r en uno, comenzando en m . Tras m iteraciones, $r_m = 0$ y la guarda es falsa.

Corrección de la postcondición.

Si salimos después de N iteraciones, $\neg G$ implica $r_N \leq 0$ e $I(N)$ implica $r_N \geq 0$. Entonces $r_N = 0$ y:

$$p_N + r_N x = mx \implies p_N = mx.$$

Regla del ciclo

Regla de corrección parcial

Si es válida $\{ I \wedge G \} C \{ I \}$, entonces es válida:

$$\{ I \} \text{ mientras } G \text{ ejecutar } C \{ I \wedge \neg G \}.$$

Si el ciclo termina, la inducción garantiza I y la condición de salida garantiza $\neg G$.

Para conectar esa regla con una especificación P, Q , necesitamos:

$$P \implies I, \quad I \wedge \neg G \implies Q.$$

Un invariante útil debe poder establecerse al inicio y aportar suficiente información para obtener Q al salir.

Teorema del invariante

Teorema del invariante del ciclo

Sea un ciclo con guarda G , precondition P y postcondición Q . El ciclo es totalmente correcto si se justifican estas cuatro propiedades.

- 1 **Propiedad base:** P implica $I(0)$ antes de la primera iteración.
- 2 **Propiedad inductiva:** para cada entero $k \geq 0$, si $I(k)$ y G son verdaderas, una iteración completa establece $I(k + 1)$.
- 3 **Eventual falsedad de la guarda:** la ejecución llega a una evaluación con G falsa en un número finito de pasos.
- 4 **Corrección de la postcondición:** si N es el menor número de iteraciones tras las cuales la guarda es falsa, $I(N) \wedge \neg G$ implica Q .

Teorema del invariante.

- 1 La propiedad base establece $I(0)$.
- 2 La propiedad inductiva permite concluir que $I(k)$ vale después de cada k iteraciones ejecutadas.
- 3 Por la tercera propiedad, existe un número finito $N \geq 0$ de iteraciones tras las cuales la guarda es falsa.
- 4 En ese estado se cumplen $I(N)$ y $\neg G$. La cuarta propiedad permite concluir Q .

La prueba también incluye $N = 0$: si la guarda es falsa desde el inicio, usamos directamente $I(0) \wedge \neg G$.

Ejemplo: ciclo

Verificaremos un ciclo que incrementa i desde cero hasta n .

Pseudocódigo

Entrada: $n \in \mathbb{Z}_{\geq 0}$

Inicializar $i \leftarrow 0$

Repetir mientras $i < n$:

$i \leftarrow i + 1$

Antes del ciclo:

$$P : n \in \mathbb{Z}_{\geq 0} \wedge i = 0.$$

Guarda $G : i < n$.

Postcondición $Q : i = n$.

$$I(k) : i = k \wedge 0 \leq k \leq n.$$

Para $n = 3$: $i = 0 \rightarrow 1 \rightarrow 2 \rightarrow 3$. Para $n = 0$, la postcondición se cumple sin ejecutar el cuerpo.

- ➊ **Propiedad base.** Al inicio, $i = 0$ y $n \geq 0$; se cumple $I(0)$.
- ➋ **Propiedad inductiva.** Si $I(k)$ y G son verdaderas, $k < n$. Después del incremento, $i = k + 1$ y $0 \leq k + 1 \leq n$: se cumple $I(k + 1)$.
- ➌ **Eventual falsedad de la guarda.** Tras n iteraciones, $i = n$, por lo que $i < n$ es falsa. Cada iteración contiene una sola asignación.
- ➍ **Postcondición.** Al salir, $\neg G$ implica $i \geq n$. El invariante garantiza $i \leq n$. Por tanto, $i = n$.

Ejemplo: multiplicación

Calculamos mx mediante sumas. Las entradas m, x permanecen fijas.

Pseudocódigo

Entrada: $m \in \mathbb{Z}_{\geq 0}, x \in \mathbb{Z}$

Inicializar $p \leftarrow 0, r \leftarrow m$

Repetir mientras $r > 0$:

$p \leftarrow p + x$

$r \leftarrow r - 1$

p acumula las sumas y r cuenta las que faltan.

Antes del ciclo:

$$P: \quad m \in \mathbb{Z}_{\geq 0} \wedge x \in \mathbb{Z} \\ \wedge p = 0 \wedge r = m.$$

Guarda $G: r > 0$.

Postcondición $Q: p = mx$.

p_k y r_k son los valores de p y r después de k iteraciones completas.

$$I(k): \quad p_k + r_k x = mx, \\ r_k \in \mathbb{Z}_{\geq 0}.$$

Lo acumulado más lo pendiente es igual al producto.

Propiedad base.

Antes de la primera iteración, $p_0 = 0$ y $r_0 = m$.

$$p_0 + r_0 x = 0 + mx = mx.$$

Como $r_0 = m \geq 0$, se cumple $I(0)$.

Consejos en el diseño de Invariantes

Un invariante se propone y después se demuestra.

- 1 **Mira el resultado que buscas.** ¿Qué información falta al salir para demostrar Q ?
Si la salida da $i \geq n$ y queremos $i = n$, necesitamos garantizar $i \leq n$.
- 2 **Busca una relación que las instrucciones conserven.** ¿Qué cambia y qué se compensa en una iteración completa?
Si p aumenta en x y rx disminuye en x , $p + rx$ no cambia. Inicialmente vale mx .
- 3 **Comprueba y ajusta tu propuesta.** ¿Se cumple al inicio? ¿Se conserva? ¿Permite concluir Q al salir?
Si no basta, añade información: en la multiplicación, $r \geq 0$ permite concluir $r = 0$ cuando la guarda es falsa.